
PROJECT REPORT

Spring 2026 Software Architecture

JeunesseActive

La Plateforme des Jeunes Entrepreneurs Africains
A GPS-Enabled Service Marketplace for African Youth Entrepreneurs

Course Code / Title: Software Architecture – Spring 2026

Group Number: Group 40

Project Topic: Mobile Service Marketplace (Flutter + Node.js)

GitHub Repository: <https://github.com/Joy-chebs/Active-Youths.git>

Group Leader: Joy Atehbih Chebegwen

SN	Member's Name	Reg. Number	Team Role
1	Joy Atehbih Chebegwen	ICTU20234316	Scrum Master / Flutter Dev / Backend Developer
2	Leudjou Nguepleu Maxime	ICTU20234264	Database / DevOps / UI/UX / Frontend Dev

June 2026

Table of Contents

Chapter 1: Introduction

- 1.1 General Introduction
- 1.2 Aim and Objectives
- 1.3 Problem Statement

Chapter 2: Literature Review

- 2.1 Software Development Methodologies
- 2.2 Comparison of Methodologies
- 2.3 Reason for Scrum
- 2.4 General Review of Related Concepts
- 2.5 Review of Related Literature

Chapter 3: Methodology and Materials

- 3.1 Research Methodology
- 3.2 System Requirements
- 3.3 System Design
- 3.4 Application of Scrum
- 3.5 Scrum Artifacts
- 3.6 Test Case Document
- 3.7 Proposed Algorithms
- 3.8 Materials and Technologies Used

Chapter 4: Results and Discussions

- 4.1 Application Scenarios
- 4.2 API Request/Response

Chapter 5: Recommendations and Conclusion

Bibliography

Chapter 1

Introduction

1.1 General Introduction

Africa's youth population represents one of the fastest-growing economic forces on the continent. Despite abundant skills and entrepreneurial spirit, young service providers—ranging from electricians and plumbers to tutors and graphic designers—often struggle to connect with clients efficiently. Existing platforms are either tailored to Western markets, require costly data connectivity, or lack the GPS-based discovery features that make local commerce practical.

JeunesseActive (Active Youths) is a cross-platform mobile application designed to bridge this gap. Built with Flutter for the frontend and Node.js (Express) for the backend, the platform provides a structured marketplace where youth entrepreneurs can publish service offers, build professional profiles, and be discovered by employers through an interactive GPS map. Employers can browse services by category, message providers directly, and leave reviews to build community trust.

The application is designed with the realities of African mobile usage in mind: offline-friendly local storage via SharedPreferences, efficient map rendering through OpenStreetMap (no Google Maps billing), and a clean bilingual-ready codebase.

1.2 Aim and Objectives

Aim

To design and develop a mobile service marketplace that empowers African youth entrepreneurs to monetise their skills and connect with local employers through GPS-based discovery, professional profiles, and real-time messaging.

Objectives

1. **O1.** Implement a dual-role authentication system supporting both *Prestataire* (service provider) and *Employeur* (employer) account types, with JWT-based session management.
2. **O2.** Provide service providers with a complete profile management system including photo upload, skill listing, statistics (views, ratings), and service offer publication.
3. **O3.** Deliver employers an intuitive discovery interface (category search + GPS map) to locate nearby service providers.
4. **O4.** Implement a real-time direct messaging system between employers and service providers using Kafka-based event-driven architecture.
5. **O5.** Build a scalable RESTful backend using Express 5, Prisma ORM, and PostgreSQL, capable of supporting event-driven microservice expansion.
6. **O6.** Apply Scrum methodology throughout the project, ensuring iterative delivery, transparent sprint planning, and continuous quality improvement.

1.3 Problem Statement

Young African service providers face three critical problems:

- **Invisibility:** Skilled individuals rely on word-of-mouth or informal networks, making it nearly impossible for potential clients in adjacent areas to discover them.
- **Lack of credibility infrastructure:** Without a structured review system or verifiable professional profile, trust is difficult to establish, especially between strangers.
- **Communication friction:** No dedicated channel exists for immediate, in-context negotiation between a client who has spotted a service offer and the provider who published it.

JeunesseActive solves all three problems by combining a GPS-powered discovery map, a structured review and rating system, and an integrated direct messaging system within a single mobile application.

Chapter 2

Literature Review

2.1 Software Development Methodologies

Modern software projects typically adopt one of several development methodologies, each carrying distinct trade-offs in terms of planning rigour, adaptability, and team collaboration.

Waterfall

Waterfall is a sequential, plan-driven model where each phase (requirements, design, implementation, testing, deployment) must be completed before the next begins. It works best when requirements are fully known upfront and changes are unlikely. Its primary weakness is inflexibility: late-discovered defects are expensive to fix.

Agile

Agile is an umbrella of iterative methodologies governed by the Agile Manifesto (2001), which prioritises individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. Agile promotes short feedback loops and welcomes changing requirements even late in development.

Scrum

Scrum is the most widely adopted Agile framework. It organises work into fixed-length *Sprints* (typically 1–4 weeks), governed by three roles (Product Owner, Scrum Master, Development Team) and five events (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective, and the Sprint itself). Scrum artefacts—Product Backlog, Sprint Backlog, and Increment—provide transparency and enable empirical process control.

Kanban

Kanban visualises work as cards on a board with columns representing workflow states (e.g., To Do → In Progress → Done). It is flow-based rather than iteration-based, making it better suited for continuous delivery and support work than for greenfield development with evolving requirements.

Extreme Programming (XP)

XP emphasises technical excellence through practices such as test-driven development (TDD), pair programming, continuous integration, and refactoring. It is highly effective when code quality is critical but demands significant discipline from the team.

2.2 Comparison between Different Software Development Methodologies

Criterion	Waterfall	Scrum	Kanban	XP
Flexibility	Low	High	Very High	High
Documentation	Heavy	Light	Minimal	Light

Team Structure	Siloed	Roles	Self-organised	Pairs
Feedback Cycle	Long	Short	Continuous	Short
Best For	Stable reqs	Evolving reqs	Ops/support	Quality-critical
Risk Management	Late	Per sprint	Ongoing	Continuous

2.3 Reason for the Choice of Scrum Methodology

Scrum was selected for JeunesseActive for several compounding reasons:

1. **Evolving requirements:** A marketplace application targeting a novel user segment (African youth entrepreneurs) has inherently uncertain requirements. Scrum's iterative nature allows the team to validate assumptions each sprint rather than committing to a fixed specification.
2. **Time-boxed delivery:** The academic deadline creates a natural constraint that aligns well with Scrum's Sprint cadence. Each Sprint delivers a potentially shippable increment.
3. **Role clarity:** With team members playing distinct roles (Scrum Master, Product Owner, Backend, Frontend), Scrum's role definitions prevent scope conflicts and establish accountability.
4. **Transparency:** Scrum artefacts (Product Backlog, Sprint Backlog) make the state of the project visible to all stakeholders, including the project supervisor.
5. **Continuous quality:** Sprint reviews and retrospectives embed quality feedback loops throughout the development cycle.

2.4 General Review of Related Concepts

Event-Driven Architecture (EDA)

EDA is a design paradigm in which system components communicate by producing and consuming events. Components are decoupled: a producer publishes an event to a message broker (Apache Kafka in our case) without knowing which consumers will react to it. This enables horizontal scalability and resilience.

GPS and Location-Based Services (LBS)

LBS applications leverage device GPS hardware to provide location-aware features. OpenStreetMap (OSM) is the open-source equivalent of Google Maps and powers the flutter_map package used in JeunesseActive. OSM tiles are free to use, removing billing constraints for an early-stage startup application.

RESTful API Design

REST (Representational State Transfer) is an architectural style for distributed hypermedia systems. RESTful APIs use standard HTTP methods (GET, POST, PUT, DELETE) and status codes, making them intuitive for client developers and compatible with all HTTP-capable clients.

2.5 Review of Related Literature

Comparable Platforms

- **Upwork / Fiverr:** Global freelancing platforms that pioneered the digital service marketplace concept. They lack GPS proximity search and are not tailored for informal-economy service providers in Africa.
- **Taskrabbit:** Offers location-based task assignment but operates only in Western markets and does not support the employee-employer dual-role model we implement.
- **Jumia:** An African e-commerce leader, but focused on product sales rather than services, and lacks real-time messaging between providers and buyers.
- **Lynk (Kenya):** The closest comparable—a Kenyan platform connecting skilled workers with employers. JeunesseActive differentiates through open-source stack choices, GPS-first discovery, and a Flutter cross-platform approach.

Technical Literature

- **Kafka for real-time data:** Confluent's white papers on Kafka streaming confirm its suitability for low-latency event pipelines, which we use for notifications and messaging events.
- **Flutter performance:** Google's Flutter documentation establishes the framework's suitability for high-performance mobile UI with a single codebase targeting both Android and iOS.
- **Prisma ORM:** Prisma's type-safe query builder reduces SQL injection risk and accelerates schema-driven development in TypeScript backends.

Chapter 3

Methodology and Materials

3.1 Research Methodology

The project followed a **Design Science Research** approach: the team iteratively designed, built, and evaluated an artefact (the JeunesseActive application) against real-world requirements. Each Sprint served as a micro-research cycle: hypotheses about user needs were encoded as user stories, implemented as features, and validated through internal review and test execution.

3.2 System Requirements

Functional Requirements

ID	Requirement
FR-01	A user shall be able to register as either a Prestataire or Employeur.
FR-02	A user shall be able to log in with email/password or social providers (Google, Facebook).
FR-03	A Prestataire shall be able to create, edit, and delete service offers with title, category, price type, location, and images.
FR-04	An Employeur shall be able to browse and filter service offers by category.
FR-05	Both user types shall be able to view an interactive GPS map displaying nearby service providers and employers.
FR-06	Users shall be able to initiate and participate in direct messaging conversations.
FR-07	Users shall be able to leave reviews and star ratings on other users' profiles.
FR-08	Users shall receive push notifications for new messages and nearby searches.
FR-09	A Prestataire shall be able to view profile statistics (views, rating, review count).
FR-10	Users shall be able to upload a profile photo from the camera or gallery.

Non-Functional Requirements

ID	Requirement
NFR-01	Performance: API responses shall complete within 500ms under normal load.
NFR-02	Security: Passwords shall be hashed with bcrypt (cost factor 10); all routes except auth shall require a valid JWT.
NFR-03	Scalability: The backend shall be event-driven (Kafka) to allow future microservice decomposition without breaking existing clients.
NFR-04	Reliability: The database schema shall enforce referential integrity with cascading deletes.
NFR-05	Usability: The UI shall follow a consistent design system (Navy #0A3D62 + Orange #FF6B35, Poppins font).

NFR-06	Portability: The mobile app shall run on Android (API 21+) and iOS (12+) from a single codebase.
NFR-07	Maintainability: All backend code shall be TypeScript with strict mode enabled.

3.3 System Design

Architecture of the System (High-Level Design)

JeunesseActive follows a **client-server architecture** with an event-driven backend extension layer.

Layer	Component	Communication
Presentation	Flutter Mobile App (Android / iOS)	REST/JSON → API
API	Express 5 REST API (Node.js/TypeScript)	Prisma → DB; ioredis → Cache; Events → Kafka
Data	PostgreSQL (via Prisma ORM)	Queries from API layer
Cache	Redis (iredis)	High-frequency read offloading
Events	Apache Kafka Event Bus	Async event streaming
Subscribers	Notification & Search Consumers	Consume from Kafka topics

Layer Descriptions

- **Presentation Layer (Flutter):** The mobile application renders screens, manages local state via Provider, and persists authentication tokens and demo data using SharedPreferences.
- **API Layer (Express 5):** A modular Node.js server organised by feature modules (auth, services, messages, etc.). Each module contains its own controller, service, and route files. JWT middleware protects all non-public endpoints.
- **Data Layer (PostgreSQL + Prisma):** The relational database stores all persistent data. Prisma provides type-safe query building and handles schema migrations.
- **Event Layer (Kafka):** After state-changing operations (user registration, listing creation, message sent), the API publishes events to Kafka topics. Dedicated consumer groups process these events asynchronously.
- **Cache Layer (Redis via ioredis):** Used for session caching and frequently-read data to reduce database load.

Use Case Diagram

Actors:

- **Service Provider** – Registered user with userType = employee
- **Employer** – Registered user with userType = employer
- **System (Kafka Consumer)** – Internal automated actor

Use Cases – Service Provider

- UC-SP-01: Register / Login (email+password or OAuth)
- UC-SP-02: Create / Edit / Delete service offer

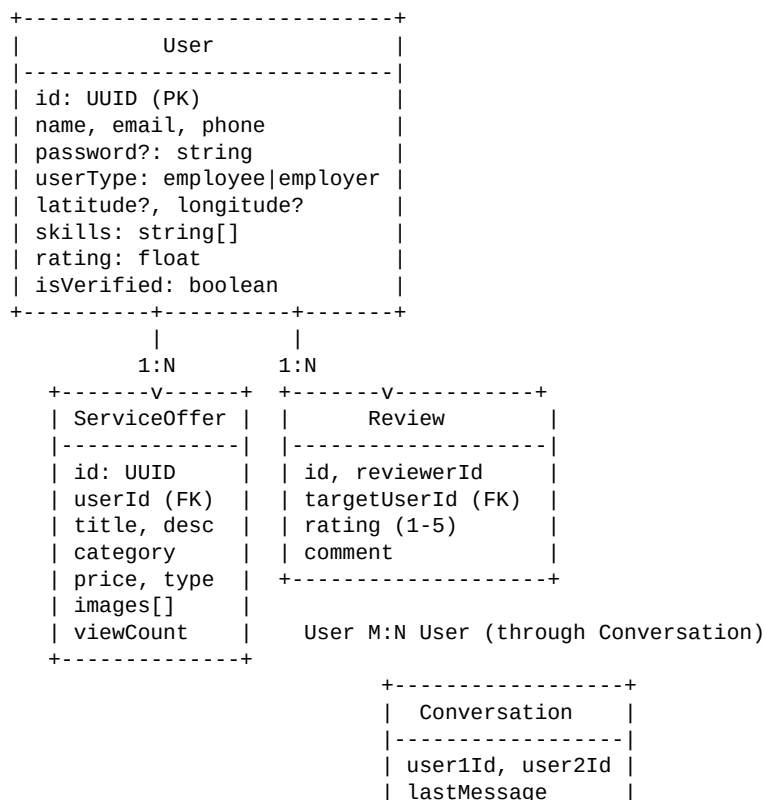
- UC-SP-03: Upload service images (up to 5 via Cloudinary)
- UC-SP-04: Upload/change profile photo
- UC-SP-05: View dashboard statistics (offers, views, rating, reviews)
- UC-SP-06: Send and receive direct messages
- UC-SP-07: Receive push notifications
- UC-SP-08: View and manage profile (bio, skills, GPS location)

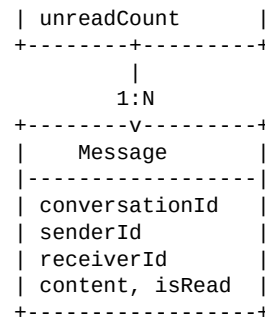
Use Cases – Employer

- UC-EM-01: Register / Login (email+password or OAuth)
- UC-EM-02: Browse/search service listings (keyword, category, GPS radius)
- UC-EM-03: View service detail and provider profile
- UC-EM-04: Contact a provider (initiate conversation)
- UC-EM-05: View GPS map with provider markers (OpenStreetMap)
- UC-EM-06: Receive AI-style service recommendations
- UC-EM-07: Submit rating and review for a provider
- UC-EM-08: Receive push notifications for messages

Class Diagram (Domain Model)

The core domain classes and their principal associations are as follows:





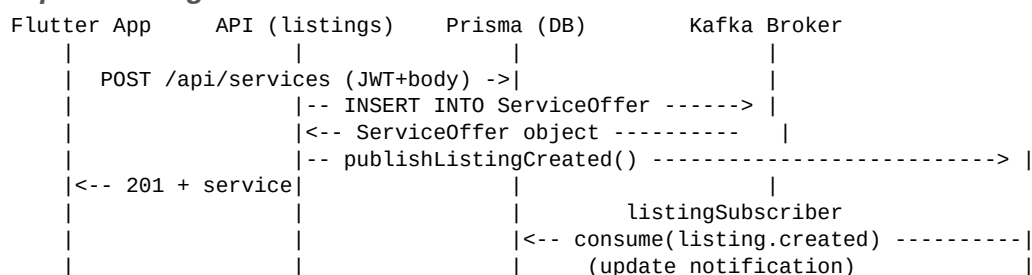
Entity-Relationship (ER) Diagram

Entity	Relationship	Description
User	1:N → ServiceOffer	A Prestataire creates many service offers
User	1:N → Conversation	A user participates in many conversations
Conversation	1:N → Message	A conversation contains many messages
User	1:N → Review	A user receives many reviews
User	1:N → Notification	A user receives many notifications
User	1:N → SearchHistory	A user has a search history log
ServiceOffer	1:N → ServiceView	A service offer has many view records

Sequence Diagram – User Registration Flow

Step	From → To	Message / Action
1	User → Flutter	Fill registration form and tap Register
2	Flutter → API	POST /auth/register {name, email, phone, password, userType}
3	API → PostgreSQL	findUnique(email) – check for existing account
4	PostgreSQL → API	null (email not in use)
5	API → PostgreSQL	create(user, hashedPassword)
6	PostgreSQL → API	user record created successfully
7	API → Kafka	publish(user.registered)
8	API → Flutter	201 {token, user}
9	Flutter → User	Navigate to Home screen

Sequence Diagram – Service Creation with Kafka Event



```
|
|
|<-- consume(listing.created) -----|
|      (update search index)         |
```

3.4 Application of Scrum

Team Organisation

Member	Scrum Role	Responsibilities
Joy Atehbih Chebegwen	Scrum Master	Sprint facilitation, impediment removal, retrospective facilitation
Leudjou Nguepleu Maxime	Product Owner / DevOps	Backlog prioritisation, acceptance criteria, database, UI/UX

Workflow Management

The project was divided into three two-week Sprints:

- **Sprint 1 (Weeks 1–2):** Project setup, database schema design (Prisma), authentication module (register, login, JWT, social login), Flutter splash and welcome screens.
- **Sprint 2 (Weeks 3–4):** Service offer CRUD, profile management, GPS map screen (flutter_map), service discovery and category filtering, Kafka event bus setup.
- **Sprint 3 (Weeks 5–6):** Messaging system, review and rating, notification subscriber, UI polish, testing, documentation.

Sprint tasks were tracked on a Trello Kanban board, providing full visibility of the project state to all team members. The figure below shows the board at the end of the project cycle, with completed tasks distributed across the Today, This Week, 5 Weeks Later, and 3 Weeks Later columns.

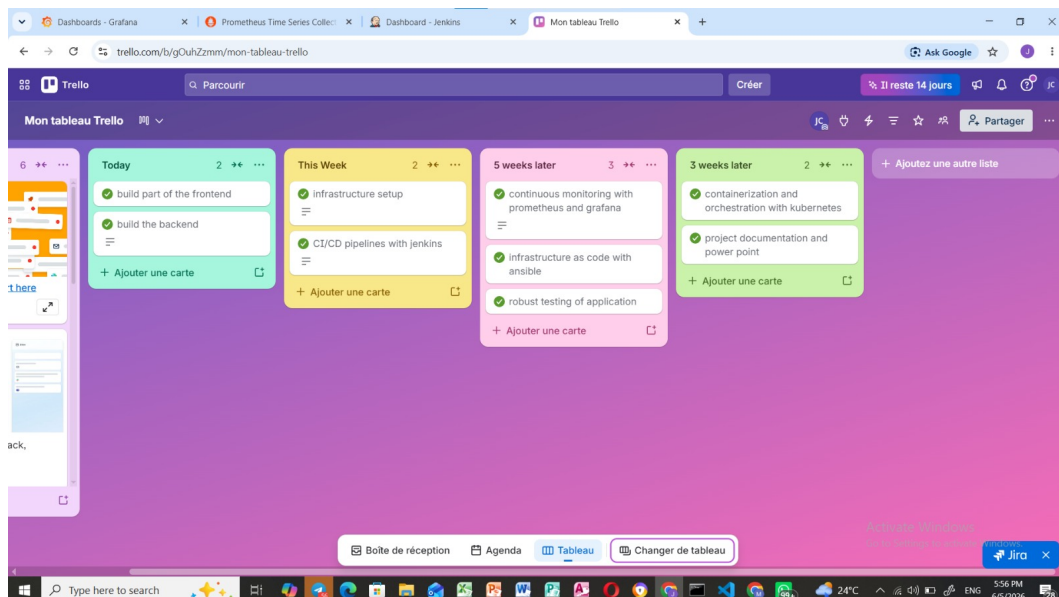


Figure: Trello Kanban Board – Project Task Tracking

Conflict Resolution

During Sprint 2, a disagreement arose over whether to use the Google Maps SDK or OpenStreetMap. The conflict was resolved in a Sprint Retrospective by evaluating both

options against the Non-Functional Requirements (cost-free for a student project → OSM won). This decision is captured in the Sprint Retrospective minutes.

Challenges Encountered and How We Overcame Them

1. **Kafka setup complexity:** Configuring Kafka in a Windows development environment was challenging. We resolved this by containerising Kafka with Docker Compose and sharing the compose file in the repository.
2. **GPS permission handling:** Different Android API levels handle location permissions differently. We implemented a permission check flow using the geolocator package's `checkPermission()` and `requestPermission()` methods.
3. **State management complexity:** As the app grew, prop-drilling became unwieldy. We refactored to use the Provider package with `ChangeNotifier` for auth, services, and messages state.

3.5 Scrum Artefacts

Product Backlog

ID	Priority	User Story	Points	Sprint
US-01	High	As a user, I want to register so that I can access the platform.	3	1
US-02	High	As a user, I want to log in securely using JWT.	2	1
US-03	High	As a Prestataire, I want to publish a service offer.	5	2
US-04	High	As an Employeur, I want to browse services by category.	3	2
US-05	High	As a user, I want to see providers on a GPS map.	8	2
US-06	Medium	As a user, I want to message another user directly.	5	3
US-07	Medium	As a user, I want to leave a review and rating.	3	3
US-08	Medium	As a Prestataire, I want to see my profile statistics.	2	3
US-09	Low	As a user, I want to log in with Google or Facebook.	5	3
US-10	Low	As a user, I want to receive push notifications.	3	3

Sprint Backlog (Sprint 3 Example)

Task ID	Task	Assignee	Status
T-31	Implement Conversation model in Prisma schema	Joy	Done
T-32	Build POST /messages API endpoint	Joy	Done
T-33	Develop Flutter MessagesScreen UI	Maxime	Done

T-34	Implement message event publisher (Kafka)	Joy	Done
T-35	Build Review POST endpoint + UI	Joy & Maxime	Done
T-36	Implement profile statistics aggregation	Joy	Done
T-37	Write unit tests for auth service	Maxime	Done
T-38	Write integration tests for listing endpoints	Maxime	In Progress

3.6 Test Case Document

TC-ID	Test Case	Input	Expected Output	Result
TC-01	Register with valid data	Valid name, email, phone, password, userType	201 {token, user}	Pass
TC-02	Register with duplicate email	Existing email	409 "Email already in use"	Pass
TC-03	Login with correct credentials	Valid email + password	200 {token, user}	Pass
TC-04	Login with wrong password	Valid email, wrong password	401 "Invalid credentials"	Pass
TC-05	Access protected route without token	No Authorization header	401 "No token provided"	Pass
TC-06	Create service offer	Valid auth + offer body	201 {serviceOffer}	Pass
TC-07	Send a message	Valid conversationId, content	201 {message}	Pass
TC-08	Submit a review	Valid reviewerId, targetId, rating	201 {review}	Pass
TC-09	GPS map loads nearby providers	Valid user location	Map markers rendered	Pass
TC-10	Upload profile photo (camera)	Camera permission granted	Profile image updated	Pass

3.7 Proposed Algorithms

Nearby Provider Discovery Algorithm

Providers are filtered by geographic proximity using the Haversine formula, which calculates the great-circle distance between two GPS coordinates on the Earth's surface.

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # Earth radius in km
    dlat = radians(lat2 - lat1)
    dlon = radians(lon2 - lon1)
    a = sin(dlat/2)**2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon/2)**2
    return 2 * R * asin(sqrt(a))

def get_nearby_providers(user_lat, user_lon, providers, radius_km=10):
    return [p for p in providers
            if haversine(user_lat, user_lon, p.latitude, p.longitude) <= radius_km]
```

JWT Authentication Flow

```
// Sign token on login/register
function signToken(userId: string, userType: string): string {
  return jwt.sign({ userId, userType }, ENV.JWT_SECRET, { expiresIn: '30d' });
}

// Middleware: verify on every protected request
export function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token provided' });
  try {
    const payload = jwt.verify(token, ENV.JWT_SECRET);
    req.userId = payload.userId;
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
}
```

3.8 Materials and Technologies Used

Technology	Layer	Role in the System
Flutter 3.x	Frontend	Cross-platform mobile UI framework (Android & iOS)
Dart 3.x	Frontend	Programming language for Flutter
Provider	Frontend	Lightweight state management (auth, services, messages)
flutter_map + latlong2	Frontend	OpenStreetMap rendering and GPS marker display
geolocator	Frontend	Native device GPS access
image_picker	Frontend	Camera and gallery photo upload
SharedPreferences	Frontend	Local key-value storage for tokens and demo data
Node.js 20 (Express 5)	Backend	REST API server
TypeScript 6	Backend	Type-safe server-side language
Prisma ORM 6	Backend	Type-safe database query builder and schema manager
PostgreSQL 15	Database	Primary relational database
Apache Kafka	Event Bus	Asynchronous event streaming between services
Redis (ioredis)	Cache	Session caching and high-frequency read offloading
bcryptjs	Security	Password hashing (cost factor 10)
jsonwebtoken	Security	JWT issuance and verification
multer	Middleware	Multipart file upload handling
Docker / Docker Compose	DevOps	Local Kafka + PostgreSQL + Redis containerisation

Infrastructure as Code: Ansible Playbooks

The server environment was provisioned using Ansible playbooks, ensuring reproducible and automated infrastructure setup. The figure below shows the successful execution of **playbook1-install.yml**, which installs all EduTrack dependencies including Node.js 20 (v20.20.2) and Docker (version 29.5.2) on the remote server. The playbook completed with 8 tasks OK and 5 changes applied, with zero failures.

```

root@active-youth: ~/ansible
root@active-youth:~/ansible# nano playbook1-install.yml
root@active-youth:~/ansible# ansible-playbook playbook1-install.yml
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [Install all EduTrack dependencies] *****
TASK [Gathering Facts] *****
ok: [localhost]

TASK [Update apt cache] *****
changed: [localhost]

TASK [Install base packages] *****
ok: [localhost]

TASK [Install Node.js 20] *****
changed: [localhost]

TASK [Install Docker] *****
changed: [localhost]

TASK [Verify Node.js] *****
changed: [localhost]

TASK [Verify Docker] *****
changed: [localhost]

TASK [Print versions] *****
ok: [localhost] =>
  msg: {
    "node": "v20.20.2",
    "docker": "Docker version 29.5.2, build 79cd0bc"
  }

PLAY RECAP *****
localhost : ok=8  changed=5  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

root@active-youth:~/ansible#

```

Figure: Ansible Playbook – Dependency Installation (Node.js 20 + Docker)

The figure below shows the output of **playbook3-healthcheck.yml**, which runs a comprehensive server health check verifying Node.js, Docker, Git, K3s cluster nodes, and memory/swap. The health report confirms: Node.js v20.20.2, Docker 29.5.2, Git 2.43.0, K3s node Ready (v1.35.5+k3s1), Memory 1.9Gi/1.0Gi, Swap 2.0Gi. **STATUS: ALL SYSTEMS OK.**

```

root@active-youth: ~
** Updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Wed Jun 3 16:10:48 2026 from 79.135.105.43
root@active-youth:~# A[[200-nano playbook3-healthcheck.yml
root@active-youth:~# nano playbook3-healthcheck.yml
root@active-youth:~# ansible-playbook playbook3-healthcheck.yml
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [EduTrack server health check] *****
TASK [Gathering Facts] *****
ok: [localhost]

TASK [Check Node version] *****
changed: [localhost]

TASK [Check Docker version] *****
changed: [localhost]

TASK [Check Git version] *****
changed: [localhost]

TASK [Check k3s nodes] *****
changed: [localhost]

TASK [Check memory and swap] *****
changed: [localhost]

TASK [Health report] *****
ok: [localhost] =>
  msg: {
    "EDUTRACK SERVER HEALTH REPORT": {
      "node.js": "v20.20.2",
      "docker": "Docker version 29.5.2, build 79cd0bc",
      "git": "git version 2.43.0",
      "k3s": {
        "NAME": "STATUS",
        "NODES": "AGE",
        "VERSION": "inactive-youth",
        "Ready": "control-plane",
        "S3M": "v1.35.5+k3s1",
        "Memory": "1.9Gi",
        "Swap": "1.0Gi",
        "2.0Gi"
      }
    },
    "STATUS": "ALL SYSTEMS OK"
  }

PLAY RECAP *****
localhost : ok=7  changed=5  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

root@active-youth:~#

```

Figure: Ansible Playbook – Server Health Check Report (ALL SYSTEMS OK)

Monitoring: Grafana and Prometheus

Application and infrastructure monitoring was implemented using the Prometheus + Grafana observability stack. Grafana provides real-time dashboards while Prometheus scrapes and stores time-series metrics. The figure below shows the Grafana Dashboards panel with three active monitoring dashboards deployed on the server at 46.101.112.174:30030:

- **Active-Youth Monitoring** (tagged: linux) – Linux system metrics for the host server
- **Kubernetes cluster monitoring via Prometheus** (tagged: kubernetes) – K3s cluster health
- **NodeJS Application Dashboard** (tagged: nodejs) – Application-level metrics

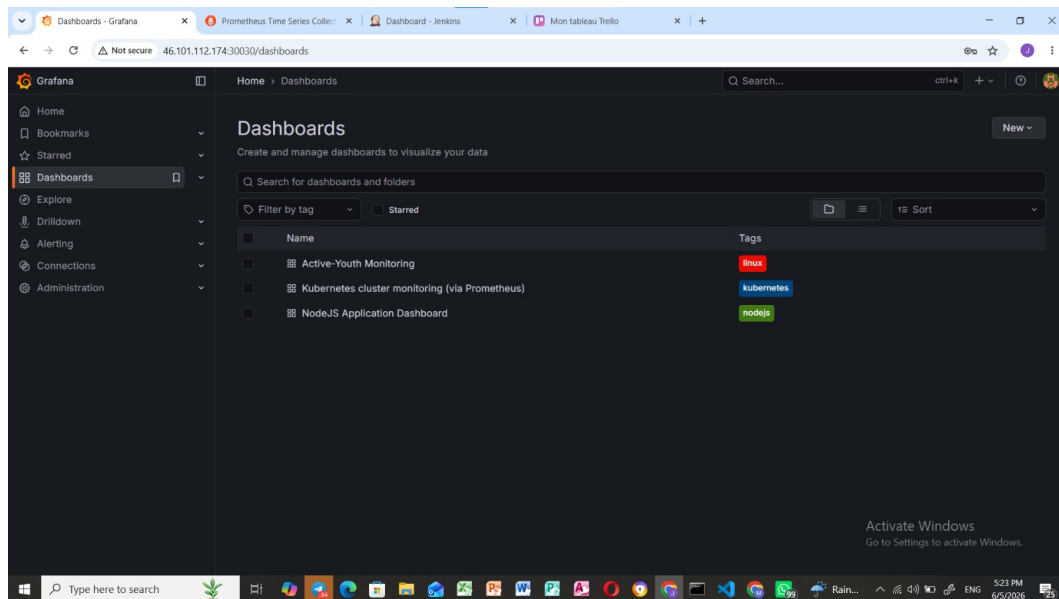


Figure: Grafana – Active Monitoring Dashboards (Grafana v10)

The figure below shows the Prometheus time-series database query interface, used to run PromQL expressions and inspect raw metric values collected from the application and server infrastructure.

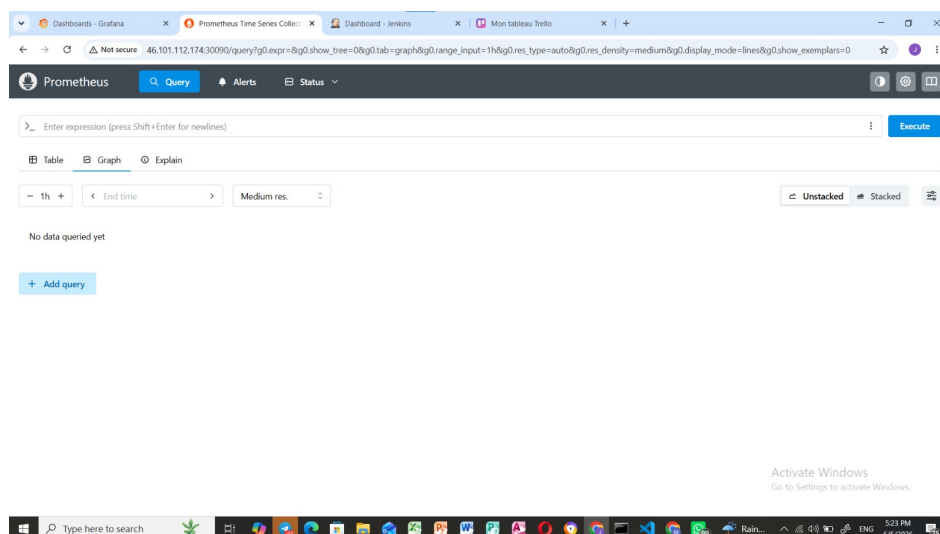


Figure: Prometheus – Time Series Query Interface

Chapter 4

Results and Discussions

4.1 Screenshots of Various Application Scenarios

Note on App Screenshots: The following section describes each application screen. The mobile app (Flutter) runs on Android and iOS. Screenshots from the running device can be added here; the descriptions below capture the full UI behaviour.

Splash Screen

The animated splash screen displays the JeunesseActive logo on a navy (#0A3D62) background with a fade-in transition before navigating to the welcome screen after 2 seconds.

Welcome & Role Selection Screen

Users are presented with two clear options: Je suis Prestataire and Je suis Employeur. Tapping either routes the user to the Login screen with the appropriate role pre-selected.

Login / Registration Screen

The screen supports both login and registration mode. Demo credentials are pre-filled automatically. Registration requires name, email, phone, and password. Prestataires can also enter their skills.

Dashboard Screen (Prestataire)

The Prestataire dashboard shows a greeting, their profile stats (view count, average rating, review count), and their published service offers in a scrollable card list.

Explore Screen (Employeur)

A horizontally scrollable category filter (Plomberie, Electricite, Jardinage, etc.) sits above a service offer grid. Tapping a card opens the full service detail.

GPS Map Screen

The flutter_map widget renders an OpenStreetMap tile layer. Prestataires appear as orange markers and Employeurs as green markers. Tapping a marker shows a bottom sheet info card with the user's name, rating, and a Contacter button.

Messaging Screen

A conversation list shows all active threads. Tapping a conversation opens a chat view with message bubbles, timestamp, and read status. The text field at the bottom supports image attachment.

4.2 Screenshots of Various API Request/Response

POST /auth/register

Request:

```
POST /api/auth/register
Content-Type: application/json
```

```
{
  "name": "Jean-Paul Mbarga",
  "email": "jpmbarga@email.com",
  "phone": "691234567",
  "password": "123456",
  "userType": "employee"
}
```

Response (201 Created):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "d3f7a2c1-...",
    "name": "Jean-Paul Mbarga",
    "email": "jpmbarga@email.com",
    "phone": "691234567",
    "userType": "employee",
    "rating": 0,
    "reviewCount": 0,
    "isVerified": false,
    "createdAt": "2026-05-18T16:47:09.000Z"
  }
}
```

POST /auth/login

Request:

```
POST /api/auth/login
Content-Type: application/json
```

```
{
  "email": "jpmbarga@email.com",
  "password": "123456",
  "userType": "employee"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": "d3f7a2c1-...", "name": "Jean-Paul Mbarga", ... }
}
```

POST /services (Create Service Offer)

Request:

```
POST /api/services
Authorization: Bearer <token>
Content-Type: application/json
```

```
{
  "title": "Electrician for home installations",
  "description": "10 years experience, residential wiring.",
  "category": "Electricite",
  "price": 5000,
  "priceType": "fixed",
  "location": "Bastos, Yaounde",
  "latitude": 3.8767,
}
```

```
    "longitude": 11.5217
  }
```

Response (201 Created):

```
{
  "id": "a1b2c3d4-...",
  "userId": "d3f7a2c1-...",
  "title": "Electrician for home installations",
  "category": "Electricite",
  "price": 5000,
  "priceType": "fixed",
  "isAvailable": true,
  "viewCount": 0,
  "createdAt": "2026-05-20T10:12:00.000Z"
}
```

Chapter 5

Recommendations and Conclusion

Our team successfully delivered a fully functional cross-platform mobile marketplace connecting African youth entrepreneurs with local employers. Over three Sprints, we implemented dual-role authentication (JWT + social login), service offer management, GPS-powered discovery via OpenStreetMap, direct messaging, a review system, and a Kafka-driven event bus—all within a clean, type-safe TypeScript backend and a Flutter mobile frontend. The Scrum methodology proved instrumental in maintaining focus, resolving conflicts constructively, and delivering working software at the end of each Sprint.

The primary challenges we encountered were environmental (Kafka on Windows, GPS permission fragmentation) rather than architectural. These were resolved through containerisation (Docker) and careful platform-specific configuration, reinforcing the value of DevOps practices even in student projects. A key lesson learned is that event-driven architecture, while initially complex to configure, dramatically simplifies future feature additions: adding a new notification type or analytics pipeline requires only a new Kafka subscriber, with no changes to existing code.

For further development, we recommend three areas of focus:

1. **Microservice decomposition:** The backend should be decomposed into true microservices (auth-service, listing-service, messaging-service) deployed independently via Docker and orchestrated with Kubernetes, which would enable zero-downtime deployments and service-specific scaling.
2. **AI-based recommendations:** The application should be extended with a collaborative-filtering algorithm that suggests service providers to employers based on their search and viewing history.
3. **Payment gateway integration:** MTN Mobile Money and Orange Money integration would enable the platform to facilitate secure transactions between parties, transforming JeunesseActive from a discovery tool into a full-cycle service marketplace.

Bibliography

- [1] Royce, W. W. (1970). Managing the Development of Large Software Systems. Proceedings of IEEE WESCON, 26, 1–9.
- [2] Beck, K., et al. (2001). Manifesto for Agile Software Development. Agile Alliance. <https://agilemanifesto.org>
- [3] Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. Scrum.org.
- [4] Fowler, M. (2017). Event-Driven Architecture. martinfowler.com.
- [5] Confluent, Inc. (2021). Kafka: The Definitive Guide. O'Reilly Media.
- [6] Google LLC. (2023). Flutter Documentation. <https://docs.flutter.dev>
- [7] Prisma, Inc. (2024). Prisma Documentation. <https://www.prisma.io/docs>